Agile Software Development Document

Nifty 50 Stock Price Forecasting using FEDformer and Comparative Time-Series Models

Project basis: This document is based on the uploaded notebook *Final_Submission_FED_FORMER_Group_5.ipynb*. The sprint plan below is a proposed Agile plan; it does not imply that the project was executed in these exact sprints.

1. Project Overview

The project develops and evaluates deep-learning approaches for short-horizon forecasting of Nifty 50 stock prices using one-minute stock-market data. The notebook downloads a Kaggle dataset, loads a subset of stock CSV files, checks missing values, engineers technical indicators, trains forecasting models, evaluates predictions, visualizes results, and includes a basic backtesting routine.

2. Project Objectives

- Prepare minute-level stock data and identify missing or problematic values.
- Enrich OHLCV inputs with technical indicators such as RSI, MACD, Bollinger Bands, and EMA.
- Implement a full FEDformer-style encoder-decoder with frequency-enhanced attention and trend/seasonal decomposition.
- Compare FEDformer with PatchTST and additional time-series model variants present in the notebook, including Autoformer-related components.
- Evaluate forecasts using MAE, RMSE, and directional accuracy, and inspect sector-level comparisons.
- Explore a simple backtest using predictions, actual prices, initial capital, and transaction-cost assumptions.

3. Agile Methodology

Use an iterative, incremental workflow with a prioritized backlog, short sprints, a working notebook increment at the end of each sprint, and review of metrics and errors. Maintain a definition of done: code runs from a clean runtime, data assumptions are documented, train/validation/test boundaries are respected, metrics are recorded, and outputs are reproducible where possible.

4. Proposed Sprint Plan

Sprint 1 — Data acquisition and audit

Install/download dependencies and the Nifty 50 one-minute dataset; enumerate available CSV files; load the selected stock subset; report row counts, null counts, and problematic columns. **Deliverable: repeatable data-loading and data-quality summary.**

Sprint 2 — Cleaning and feature engineering

Standardize column names and time ordering; handle missing/invalid values; construct OHLCV features and technical indicators (RSI, MACD, Bollinger Bands, EMA); verify feature columns and target alignment. **Deliverable: validated feature-ready time series.**

Sprint 3 — Dataset and split pipeline

Build sequence-to-horizon datasets; scale features using training data only; create train/validation/test subsets; verify sequence lengths, prediction targets, and no temporal leakage. **Deliverable: reusable dataset and DataLoader pipeline.**

Sprint 4 — FEDformer implementation

Implement decomposition blocks, frequency-enhanced attention, encoder/decoder layers, and forecast heads; train with optimizer, learning-rate scheduling, validation loss, and early stopping. **Deliverable: trained FEDformer checkpoint and training history.**

Sprint 5 — Baselines and comparison

Train PatchTST and other model variants implemented in the notebook under comparable data splits and forecast horizons; collect predictions and metrics. **Deliverable: comparable model result table.**

Sprint 6 — Evaluation and reporting

Calculate MAE, RMSE, and directional accuracy; create sector/model comparison plots; run the exploratory backtest; document limitations and prepare the final report. **Deliverable: result tables, plots, and documented findings.**

5. Product Backlog and Acceptance Criteria

Backlog item	Acceptance criteria
Data ingestion	Notebook downloads or locates the dataset, loads the selected CSV files, and reports row counts and missing values.
Feature engineering	OHLCV and requested technical indicators are created with documented handling of indicator warm-up NaNs.
Forecast dataset	Inputs and targets have expected shapes; temporal order is preserved; scaling is fit on training data only.
Model training	Training and validation losses are logged; best validation state is retained where implemented; early stopping is checked.
Evaluation	Test metrics are calculated in original price units where intended and are labeled by stock, model, horizon, and sector where available.
Visualization/backtest	Plots use actual generated results; backtest assumptions and transaction costs are stated; results are not presented as investment advice.

6. System Workflow

Kaggle data download → CSV loading and data-quality checks → chronological cleaning → OHLCV and technical-indicator engineering → feature scaling and sequence creation → time-aware train/validation/test split → FEDformer and baseline model training → prediction and metric calculation → plots, sector comparisons, and exploratory backtesting.

7. Technology Stack

- Python and Jupyter Notebook
- Pandas and NumPy for data handling
- KaggleHub for dataset retrieval
- PyTorch for neural-network models and DataLoaders
- Scikit-learn for scaling and evaluation utilities
- ta for technical indicators
- Matplotlib for result visualization

8. Testing and Quality Assurance

- Data tests: expected columns, timestamp ordering, duplicates, nulls, and numeric types.
- Feature tests: indicator values and handling of warm-up rows.
- Sequence tests: input/target dimensions and correct future-horizon alignment.
- Split tests: chronological separation and no scaler fitting on validation/test data.
- Training tests: finite losses, checkpoint restoration, and consistent device handling.
- Evaluation tests: metrics computed on aligned predictions and targets in the intended scale.
- Reproducibility tests: record random seeds, library versions, selected stocks, and configuration.

9. Risks and Mitigations

- Market noise and non-stationarity: evaluate across multiple stocks and time periods; avoid claiming guaranteed predictive power.
- Data leakage: preserve chronological splits and fit preprocessing only on training data.
- Missing data or inconsistent CSV schemas: validate every file and document exclusions.

- Unequal model comparison: use the same test windows and comparable forecast horizons for each model.
- Compute limitations: use a documented subset of stocks and configurable batch size, sequence length, and epochs.
- Backtest over-interpretation: account for transaction costs and clearly label the backtest as experimental.

10. Configuration Note

The notebook's introductory markdown describes several intended prediction horizons, while the later active configuration sets `PRED_HORIZONS = [12]`. The report should therefore state the horizon actually used in the executed run. The code should also be checked to confirm that the implemented train/validation/test split matches the planned proportions.

11. Definition of Done

- Notebook executes end-to-end in a clean environment with documented dependencies.
- Dataset selection and cleaning rules are recorded.
- Models are compared on a consistent, leakage-free test set.
- Metrics, plots, and configuration are saved or reproducibly generated.
- Limitations and financial-risk caveats are included in the project report.

12. Conclusion

An Agile approach supports incremental development of the Nifty 50 forecasting pipeline, separating data validation, feature engineering, model implementation, comparative evaluation, and reporting into reviewable increments. Model quality should be judged from verified out-of-sample results rather than architecture complexity alone.